

SUPPLEMENTS

This is the mathematical supplements to the scratch-built RHF/SCF engine in this repository.

Every section below corresponds to one function or class in the codebase

This is a from-scratch, first-principles, proof-of-concept-focused implementation.

Every integral is evaluated via explicit Obara-Saika recursion in pure NumPy, with no primitive screening or batching. It is not meant to be fast, it is meant to be a direct, line-by-line map from the equations below to working code.

1. Gaussian normalization: `shell.__init__` (`mol_basis_builder.py`)

Every contracted Cartesian basis function is a normalized sum of primitives:

$$\phi(\mathbf{r}, \mathbf{r}_A, \alpha_p, c_p) = N_{\text{cont}} \sum_p c_p N_{\text{prim}}(\alpha_p) P(\mathbf{r} - \mathbf{r}_A) \exp(-\alpha_p ||\mathbf{r} - \mathbf{r}_A||^2)$$

where ' $P(\mathbf{r} - \mathbf{r}_A)$ ' is the equivalent to the polynomial ' $P(x, y, z) = x^{n_x} y^{n_y} z^{n_z}$ ', and ' $n_x + n_y + n_z = l$ ', the total angular momentum.

Contraction normalization (self-overlap of the full contracted function = 1), using the primitive-pair overlap matrix:

$$S_{pq} = \left(\frac{2\sqrt{\alpha_p \alpha_q}}{\alpha_p + \alpha_q} \right)^{l+3/2}$$

:

$$N_{\text{cont}} = \left(\sum_{p,q} c_p c_q S_{pq} \right)^{-1/2}$$

`effective_coef` returns $N_{\text{cont}} c_p N_{\text{prim}}(\alpha_p)$, which is the single scalar multiplied into every integral involving this primitive in `matrix_builders.py`.

Primitive normalization (self-overlap of a single primitive = 1), for the "pure" axial component ($l, 0, 0$):

$$N_{\text{prim}}(\alpha) = \left(\frac{2\alpha}{\pi} \right)^{3/4} (4\alpha)^{l/2} / \sqrt{(2l-1)!!}$$

' α_p ' and ' c_p ' are fetched from Basis Set Exchange, while both normalization are calculated following these formulas.

NOTE (angular normalization): the $1/\sqrt{(2l-1)!!}$ factor above is exact only for the "pure" Cartesian components where all angular momentum sits on one axis, for example $(2, 0, 0)$, $(0, 2, 0)$, $(0, 0, 2)$ for d functions. The general per-component formula, for Cartesian indices (n_x, n_y, n_z) , is

$$\frac{1}{\sqrt{(2n_x-1)!!(2n_y-1)!!(2n_z-1)!!}}$$

which reduces to $1/\sqrt{(2l-1)!!}$ for pure components but gives exactly 1 for mixed components like $(1, 1, 0)$, not $1/\sqrt{3}$. `shell` applies one `angular_norm` value per shell (for all combinations, computed from l alone), so mixed Cartesian components (' d_{xy} ', ' d_{xz} ', ' d_{yz} ' and higher- l analogues) are technically wrong. However, this normalization scheme (or any normalization scheme) leaves the SCF energy unaffected (see **Roothan-Hall equations** section) while still changing individual AO-matrix elements for any basis with $l \geq 2$ functions (verified directly against Psi4 reference matrices for cc-pVDZ). The normalization scheme, both primitive and contraction, was attempted to replicate Psi4's coefficients with already baked-in normalization. cc-pVDZ, or higher basis sets, have more specialized schemes, and currently fully replicating them is not a priority of this repository.

Only the "superdiagonal" functions like or are normalized in Cartesian. The other functions have well-defined fractional normalizations that are typically accounted for automatically in the orthonormalization of the basis within the SCF code - *NVIDIA cuEST Documentation*

Source: [IOData Documentation](#), [NVIDIA cuEST Documentation](#)

2. Matrices construction, Roothaan-Hall equations, and symmetric orthogonalization: roothan_solver (roothan_solver.py)

The Hamiltonian matrix with no electron interaction is constructed directly from the kinetic integral matrix and the nuclear attraction (potential) integral matrix: $\mathbf{H} = \mathbf{T} + \mathbf{V}$. This is also used as the initial guess for the Fock matrix in this implementation, although there are different guess strategies.

Closed-shell density matrix from the occupied MO block ' $\mathbf{C}_{\text{occ}}$ ' (n_{docc} columns):

$$D_{\mu\nu} = \sum_i^{\text{occ}} C_{\mu i} C_{\nu i} = (\mathbf{C}_{\text{occ}} \mathbf{C}_{\text{occ}}^T)_{\mu\nu}$$

Coulomb and exchange matrices, contracted directly against the full 4-index

ERI tensor $I_{pqrs} = (pq|rs)$:

$$J_{pq} = \sum_{rs} I_{pqrs} D_{rs}, \quad K_{pq} = \sum_{rs} I_{prqs} D_{rs}, \quad \mathbf{F} = \mathbf{H} + 2\mathbf{J} - \mathbf{K}$$

implemented as `np.einsum('pqrs,rs->pq', I, D)` and `np.einsum('prqs,rs->pq', I, D)`. Notice the index permutation between the two einsum strings (q, r swap roles) is exactly what distinguishes the exchange integral from the Coulomb integral.

The RHF equations in a non-orthogonal AO basis are a generalized eigenvalue problem,

$$\mathbf{FC} = \mathbf{SC}\epsilon$$

To eliminate $\mathbf{S}$ and make this an ordinary eigenvalue problem, we need to find a transformation matrix $\mathbf{X}$ such that $\mathbf{X}^T \mathbf{S} \mathbf{X} = \mathbf{I}$. Symmetric (Löwdin) orthogonalization does this by using

$$\mathbf{X} = \mathbf{S}^{-1/2} = \mathbf{U} \mathbf{S}^{-1/2} \mathbf{U}^T$$

and $\mathbf{s}$ can be obtained by diagonalizing the overlap matrix, $\mathbf{S} = \mathbf{U} \mathbf{s} \mathbf{U}^T$, which is well-defined only because $\mathbf{S}$ is symmetric positive (semi)definite, making its eigenvalues positive and the elementwise inverse-square-root valid.

With $\mathbf{F}' = \mathbf{X}^T \mathbf{F} \mathbf{X}$ and $\mathbf{C} = \mathbf{X} \mathbf{C}'$, the transformed problem $\mathbf{F}' \mathbf{C}' = \mathbf{C}' \epsilon$ is ordinary and can be solved trivially.

The RHF energy can be obtained via constructed matrices:

$$E_0 = \sum_{\mu\nu} D_{\mu\nu} (H_{\mu\nu} + F_{\mu\nu}) + V_{NN}$$

Source: [Szabo & Ostlund, Ch. 3.4]

Why different shell-level angular normalization schemes doesn't break the energy:

The generalized eigensolve above enforces $\mathbf{C}^T \mathbf{S} \mathbf{C} = \mathbf{I}$ using the $\mathbf{S}$ constructed from the AO basis, so even if any given AO is scaled by some constant, that scaling is automatically absorbed into the corresponding MO coefficients. The physical molecular orbitals and total energy come out identical either way. What is not identical are the electron integrals themselves, confirmed directly for cc-pVDZ, where individual S, T, V, I entries differ from Psi4's reference values while the converged SCF energy still matches to $\sim 10^{-8} E_h$. (See NVIDIA cuEST Documentation above)

3. DIIS convergence acceleration: diis (roothan_solver.py)

The DIIS residual error vector, which vanishes at convergence, is defined as:

$$\mathbf{e}_i = \mathbf{F}_i \mathbf{D}_i \mathbf{S} - \mathbf{S} \mathbf{D}_i \mathbf{F}_i$$

We can represent the current residual vector $\mathbf{e}_{m+1}$ as a linear combination of previous residual vectors:

$$\mathbf{e}_{m+1} = \sum_i^m c_i \mathbf{e}_i$$

By minimizing $\|\sum_i c_i \mathbf{e}_i\|^2$ to approximate the zero vector in the least-square sense under the constraint $\sum_i c_i = 1$, a Lagrange-multiplier problem, which will lead to the construction of the matrix equation:

$$\begin{pmatrix} B_{11} & \cdots & B_{1n} & -1 \\ \vdots & \ddots & \vdots & \vdots \\ B_{n1} & \cdots & B_{nn} & -1 \\ -1 & \cdots & -1 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ \vdots \\ c_n \\ \lambda \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ -1 \end{pmatrix}, \quad B_{ij} = \mathbf{e}_i \cdot \mathbf{e}_j$$

`e_array @ e_array.T` computes every pairwise error-vector dot product B_{ij} in a single matrix multiply. The extrapolated Fock matrix, $\mathbf{F}_{\text{new}} = \sum_i c_i \mathbf{F}_i$, is built via `np.einsum('i,ijk->jk', coeffs, F_history)`. A sliding 8-iteration history window keeps the DIIS subspace from being polluted by early, poorly-converged iterations.

Source: [Sherrill group, "Some Comments on Accelerating Convergence of Iterative Sequences Using Direct Inversion of the Iterative Subspace \(DIIS\)"](#)

4. Mulliken Population Analysis: `calc_mulliken_charges` (`roothan_solver.py`)

Mulliken analysis partitions the total electron density among atoms by assigning each basis function's population to the atom on which it is centered. The population matrix is defined as:

$$\mathbf{P} = 2\mathbf{D}\mathbf{S}$$

where the factor of 2 accounts for double occupancy of the restricted spatial orbitals. The trace recovers the total electron count, $\sum_\mu P_{\mu\mu} = N_{\text{elec}}$, so the diagonal element $P_{\mu\mu}$ is interpreted as the number of electrons associated with basis function μ . `np.diag(P)` extracts these AO populations.

The total (electron) population on atom A is the sum over all basis functions centered on that atom:

$$e_A = \sum_{\mu \in A} P_{\mu\mu}$$

`basis.function_to_center(ao_idx)` maps each basis function to its parent atom, and the loop accumulates $P_{\mu\mu}$ into the corresponding atomic total.

The Mulliken charge is the difference between the nuclear charge and the electrons assigned to the atom:

$$Q_A = Z_A - e_A$$

By construction, $\sum_A Q_A$ equals the total molecular charge, and $\sum_A e_A$ equals the total number of electrons.

Source: [Szabo & Ostlund, Ch. 3.4]

5. Overlap integrals: `get_overlap_integrals` (`integral_solvers.py`)

Definition

The two-center overlap integral between two primitive Cartesian Gaussians is

$$S_{ab} = (\mathbf{a}|\mathbf{b}) = \int d\mathbf{r} \phi_a(\mathbf{r}) \phi_b(\mathbf{r})$$

Notations

For two primitives on centers $\mathbf{A}$ (exponent a) and $\mathbf{B}$ (exponent b), the Gaussian Product Theorem gives a single Gaussian on the weighted center $\mathbf{P}$:

$$\zeta = a + b, \quad \xi = \frac{ab}{\zeta}, \quad \mathbf{P} = \frac{a\mathbf{A} + b\mathbf{B}}{\zeta}$$

$$e^{-a|\mathbf{r}-\mathbf{A}|^2} e^{-b|\mathbf{r}-\mathbf{B}|^2} = K_{AB} e^{-\zeta|\mathbf{r}-\mathbf{P}|^2}, \quad K_{AB} = e^{-\xi|\mathbf{A}-\mathbf{B}|^2}$$

All recursions below are the Obara-Saika (OS) vertical recurrence relations built on this product Gaussian.

Decoupling S_{base} (a math trick *and* a code trick)

The s-s base case, where both functions with zero angular momentum, is the Gaussian product prefactor:

$$S_{\text{base}} = S_{00} = \left(\frac{\pi}{\zeta}\right)^{3/2} \cdot e^{-\xi|\mathbf{A}-\mathbf{B}|^2}$$

This factor does not depend on the individual angular momentum indices, i.e. ' n_{ax} '. Every higher-angular-momentum overlap is ' S_{base} ' multiplied by a dimensionless, purely angular-momentum-dependent factor. This was exploited by splitting the work: `get_overlap_integrals` returns only the dimensionless table $\tilde{S}$ (seeded at ' $\tilde{S}_{00} = 1$ '), and ' S_{base} ' is computed once per primitive pair up in `matrix_builders.py`. This is a **code trick** (compute the prefactor once, not once per angular component) as much as a **math trick** (the recursion tables are cleaner when stripped of the exponent-dependent constant).

Independence of Cartesian axes (and why NAI/ERI can't do this)

Since there is no operator coupling the three Cartesian directions, the Gaussian product therefore factorizes cleanly, and the 3D overlap is an exact product of three independent 1D tables:

$$S_{ab} = S_{\text{base}} \tilde{S}_{n_{ax}n_{bx}}^{(x)} \tilde{S}_{n_{ay}n_{by}}^{(y)} \tilde{S}_{n_{az}n_{bz}}^{(z)}$$

This separability is extremely important for overlap integrals to be computed easily, and it is **exactly what nuclear attraction (6.) and electron repulsion (7.) lack**: the $1/r$ Coulomb operator is not separable across Cartesian axes, so those integrals cannot be written as a product of independent 1D tables and instead require a shared Boys-function index coupling all axes together.

The OS recursion

$$\begin{aligned} \tilde{S}_{i,0}^{(x)} &= (P_x - A_x) \tilde{S}_{i-1,0} + \frac{i-1}{2\zeta} \tilde{S}_{i-2,0} \\ \tilde{S}_{i,j}^{(x)} &= (P_x - B_x) \tilde{S}_{i,j-1} + \frac{i}{2\zeta} \tilde{S}_{i-1,j-1} + \frac{j-1}{2\zeta} \tilde{S}_{i,j-2} \end{aligned}$$

Index convention: $\tilde{S}_{i,j}^{(x)}$ is the dimensionless overlap along the x -axis between an x -angular-momentum of i on center A and j on center B. The code indexes the table as `SI[axis, i, j]` with `axis` in $\{0, 1, 2\}$ vectorizing x, y, z simultaneously, since $\mathbf{P} - \mathbf{A}$ and $\mathbf{P} - \mathbf{B}$ are 3-vectors and the recursion is identical along each axis.

Note the recursion below is written in "step-down" form (' $\tilde{S}_{i,j}$ ' in terms of ' $\tilde{S}_{i-1,j}$ ') rather than the "step-up" form (' $\tilde{S}_{i+1,j}$ ', ' $\tilde{S}_{i,j+1}$ ') used in the original Obara-Saika paper. The two are algebraically identical; the step-down indexing reads more naturally when filling a table from the origin outward, which is what the code does.

The algorithm: We fill a $(l_a+1) \times (l_b+1)$ table of angular-momentum index pairs. The code fills it in two phases, matching the two relations above:

1. **The $j = 0$ column** ($\tilde{S}_{i,0}$) is filled first by pure center-A stepping, each entry depends only on lower- i entries in the same column.
2. **The $j > 0$ entries** ($\tilde{S}_{i,j}$) are then filled by center-B stepping, each depending on entries already computed in the $j - 1$ and $j - 2$ columns.

A source term is 0 whenever its index would go negative.

Source: [S. Obara and A. Saika, J. Chem. Phys 84, 3963 \(1986\)](#)

6. Kinetic energy integrals: `get_kinetic_integrals` (`integral_solvers.py`)

Definition

The two-center kinetic energy integral is

$$T_{ab} = (\mathbf{a} | -\frac{1}{2} \nabla^2 | \mathbf{b}) = -\frac{1}{2} \int d\mathbf{r} \phi_a(\mathbf{r}) \nabla^2 \phi_b(\mathbf{r})$$

Decoupling T_base, and why it is *not* a flat product

Like overlap, the kinetic integral shares the same ‘ S_{base} ’ Gaussian prefactor, and `get_kinetic_integrals` returns a dimensionless per-axis table $\tilde{T}$ seeded so the s-s case is correct relative to ‘ S_{base} ’. The seed is the 1D s-s kinetic shape factor per axis:

$$\tilde{T}_{0,0}^{(x)} = \xi(1 - 2\xi(A_x - B_x)^2)$$

The difference from overlap. The kinetic operator is the Laplacian, a sum

of three second derivatives. By the product rule, differentiating one Cartesian axis leaves the other two axes as **overlap integrals**. The 3D kinetic integral is therefore a sum of three cross terms (Helgaker 2000, eq. 9.3.38):

$$T_{ab} = S_{\text{base}} \left[\tilde{T}^{(x)} \tilde{S}^{(y)} \tilde{S}^{(z)} + \tilde{S}^{(x)} \tilde{T}^{(y)} \tilde{S}^{(z)} + \tilde{S}^{(x)} \tilde{S}^{(y)} \tilde{T}^{(z)} \right]$$

(indices ‘ $n_{ax}n_{bx}$ ’ etc. suppressed for readability). Writing it as ‘ $S_{\text{base}}\tilde{T}^{(x)}\tilde{T}^{(y)}\tilde{T}^{(z)}$ ’ is **incorrect**. It passes s/p-only validation by coincidence (the cross terms and the flat product happen to agree when at most one axis carries angular momentum) and only fails once $l \geq 2$ appears on a mixed component. This assembly lives in `build_s_t_v` (`matrix_builders.py`), which is why `get_kinetic_integrals` returns only the per-axis $\tilde{T}$ pieces and depends on the overlap table $\tilde{S}$ as an input.

Kinetic is not *fully* separable like overlap. Each individual term in the sum above is a product of one kinetic axis and two overlap axes, so the algorithm is still per-axis 1D tables, only the **assembly** changes.

The OS recursion

$$\begin{aligned} \tilde{T}_{i,0} &= X_{PA} \tilde{T}_{i-1,0} + \frac{i-1}{2\zeta} \tilde{T}_{i-2,0} + 2\xi \left[\tilde{S}_{i,0} - \frac{i-1}{2a} \tilde{S}_{i-2,0} \right] \\ \tilde{T}_{i,j} &= X_{PB} \tilde{T}_{i,j-1} + \frac{i}{2\zeta} \tilde{T}_{i-1,j-1} + \frac{j-1}{2\zeta} \tilde{T}_{i,j-2} + 2\xi \left[\tilde{S}_{i,j} - \frac{j-1}{2b} \tilde{S}_{i,j-2} \right] \end{aligned}$$

The algorithm. Structurally identical to overlap: a $(l_a+1) \times (l_b+1)$ table filled in two phases (the $j = 0$ column by center-A stepping, then $j > 0$ by center-B stepping). The only addition is the bracketed final term that requires the overlap table. This is why `get_overlap_integrals` is always called before `get_kinetic_integrals`.

Note. This scheme (building $\tilde{T}$ as a correction on top of $\tilde{S}$, with the sum-of-cross-terms 3D assembly one layer up) was assembled from the general OS relations rather than copied from any single source, and these were verified matching Psi4’s `mintools.ao_kinetic()` with s/p/d/f test cases.

Source: [S. Obara and A. Saika, J. Chem. Phys 84, 3963 \(1986\)](#)

7. The Boys function: `boys_function` (`integral_solvers.py`)

Every integral involving the $1/r_{12}$ Coulomb operator against Gaussians uses the Boys function

$$F_m(x) = \int_0^1 t^{2m} e^{-xt^2} dt$$

This implementation evaluates it via the gamma function (`scipy.special.gamma`) and the regularized incomplete gamma function (`scipy.special.gammainc`) (Helgaker 2000, eq. 9.8.21):

$$F_m(x) = \frac{\Gamma(m + \frac{1}{2})}{2x^{m+1/2}} P(m + \frac{1}{2}, x), \quad P(a, x) = \frac{\gamma(a, x)}{\Gamma(a)}$$

At the limit $x \rightarrow 0$ branch, the code uses the direct limit $F_m(0) = \frac{1}{2m+1}$ when $x < 1e-9$; the formula above is 0/0 indeterminate at $x = 0$.

[Original definition: S. F. Boys, *Proc. R. Soc. Lond. A* **200**, 542 (1950).]

Original definition: [S. F. Boys, Proc. R. Soc. London A 200, 542 \(1950\)](#)

8. Nuclear attraction integrals: `get_NAIs` (`integral_solvers.py`)

Definition

The one-electron nuclear attraction integral between two primitives and a point charge at nucleus **C** is

$$V_{ab} = \left(\mathbf{a} \left| \frac{1}{|\mathbf{r} - \mathbf{C}|} \right| \mathbf{b} \right) = \int d\mathbf{r} \frac{\phi_a(\mathbf{r})\phi_b(\mathbf{r})}{|\mathbf{r} - \mathbf{C}|}$$

The axes no longer separate

Overlap and kinetic factor into per-axis 1D tables because their operators do not couple the Cartesian directions. The Coulomb operator $1/|\mathbf{r} - \mathbf{C}|$ **does**. It is a function of the full 3D distance and cannot be split into independent Cartesian axes. Instead, the coupling is handled by introducing an **auxiliary index** m , the Boys function order, which threads through all three axes simultaneously. This is why the working array is 7-dimensional $(n_{ax}, n_{ay}, n_{az}, n_{bx}, n_{by}, n_{bz}, m)$, rather than three separate 2D tables.

The Boys function seed

The recursion is seeded on the Boys function with argument $U = \zeta|\mathbf{P} - \mathbf{C}|^2$:

$$[\mathbf{0}|\mathbf{0}]^{(m)} = F_m(U), \quad m = 0, 1, \dots, l_a + l_b$$

Only the $m = 0$ value is physically meaningful in the final integral; the higher- m values only serves as a "scaffold" as the recursion climbs in angular momentum. This is why the base case must seed *all* orders $m = 0 \dots l_a + l_b$ up front, and why the code reads out only `vi[ ..., 0]`.

The V_{base} prefactor and the nuclear charge

The Gaussian/Coulomb prefactor is ' $V_{\text{base}} = 2\sqrt{\zeta/\pi}S_{\text{base}}$ '. It's a prefactor independent of angular momentum, so it's computed outside the function. The final matrix element also carries the nuclear charge ' $-Z_C$ ', summed over all nuclei **C**. These terms are multiplied in `build_S_T_V`.

The OS recursion

Stepping up the x -index on center A has the following recursion:

$$[\mathbf{a}+1_x|\mathbf{b}]^{(m)} = (P_x - A_x)[\mathbf{a}|\mathbf{b}]^{(m)} - (P_x - C_x)[\mathbf{a}|\mathbf{b}]^{(m+1)} + \frac{a_x}{2\zeta} \left([\mathbf{a}-1_x|\mathbf{b}]^{(m)} - [\mathbf{a}-1_x|\mathbf{b}]^{(m+1)} \right) + \frac{b_x}{2\zeta} \left([\mathbf{a}|\mathbf{b}-1_x]^{(m)} - [\mathbf{a}|\mathbf{b}-1_x]^{(m+1)} \right)$$

The other five spatial directions follow the identical pattern with the appropriate displacement vector.

The algorithm: Unlike the overlap table, we cannot fill this axis-by-axis independently, the m -coupling forces a specific global order. The code fills the table shell by shell in total angular momentum $L = 1, 2, \dots, l_a + l_b$. For each L , it visits only the index combinations whose spatial indices sum to exactly L , and for each such combination fills all needed orders $m = 0 \dots (l_a + l_b - L)$. Building strictly in increasing L guarantees every lower- L dependency already exists before it is needed. Within a combination, the code steps up the *first* nonzero spatial index it finds (the six `if/elif` branches), since any valid step-down direction yields the same result.

Index conventions and the shift helper

The 7-index recursion has too many boundary conditions to write as explicit `if` branches per term. The pattern is also regular: in each term only one or two indices are shifted. `shift_NAI({axis: delta, ...})` takes the current loop indices, applies the requested shifts, and returns 0 automatically if any index would go negative. This keeps each recursion term readable as a direct transcription of the OS formula.

Source: [S. Obara and A. Saika, J. Chem. Phys 84, 3963 \(1986\)](#)

9. Electron repulsion integrals: `get_ERIs` (`integral_solvers.py`)

Definition

The two-electron repulsion integral over four primitives, in chemist's notation, is

$$(\mathbf{ab}|\mathbf{cd}) = \iint d\mathbf{r}_1 d\mathbf{r}_2 \frac{\phi_a(\mathbf{r}_1)\phi_b(\mathbf{r}_1)\phi_c(\mathbf{r}_2)\phi_d(\mathbf{r}_2)}{|\mathbf{r}_1 - \mathbf{r}_2|}$$

The four-center generalization of NAI

The ERI is the nuclear attraction integral taken to extreme. The same non-separability applies, so the same m -index is needed, but now over four centers instead of two. This is the origin of the **13-dimensional** working array: three spatial indices on each of the four centers, plus the shared Boys order m .

Notations

Each electron's pair collapses to a Gaussian-product center (as in §1):

$$\zeta = a + b, \quad \mathbf{P} = \frac{a\mathbf{A} + b\mathbf{B}}{\zeta}; \quad \eta = c + d, \quad \mathbf{Q} = \frac{c\mathbf{C} + d\mathbf{D}}{\eta}$$

The two products then couple through a **combined center** $\mathbf{W}$ and reduced exponent ρ :

$$\mathbf{W} = \frac{\zeta\mathbf{P} + \eta\mathbf{Q}}{\zeta + \eta}, \quad \rho = \frac{\zeta\eta}{\zeta + \eta}$$

$\mathbf{W} - \mathbf{P}$ and $\mathbf{W} - \mathbf{Q}$ in ERI play a similar role to $\mathbf{P} - \mathbf{C}$ in NAI.

The Boys function seed and prefactor

Seeded on the Boys function with argument $T = \rho|\mathbf{P} - \mathbf{Q}|^2$:

$$[00|00]^{(m)} = F_m(T), \quad m = 0, \dots, l_a + l_b + l_c + l_d$$

with prefactor $\text{ERI}_{\text{base}} = 2\sqrt{\rho/\pi} S_{\text{base}}^{(ab)} S_{\text{base}}^{(cd)}$, the product of both pairs' Gaussian prefactors. As in NAI, the code only reads the final integral, the $m = 0$ case: `ERI_raw[... , 0]`.

The OS recursion

Stepping up the x -index on center A has the following recursion:

$$\begin{aligned} [\mathbf{a}+1_x\mathbf{b}|\mathbf{cd}]^{(m)} &= (P_x - A_x)[\dots]^{(m)} + (W_x - P_x)[\dots]^{(m+1)} \\ &+ \frac{a_x}{2\zeta} \left([\mathbf{a}-1_x\dots]^{(m)} - \frac{\eta}{\zeta+\eta} [\mathbf{a}-1_x\dots]^{(m+1)} \right) + \frac{b_x}{2\zeta} (\dots) \\ &+ \frac{c_x}{2(\zeta+\eta)} [\dots\mathbf{c}-1_x\dots]^{(m+1)} + \frac{d_x}{2(\zeta+\eta)} [\dots\mathbf{d}-1_x]^{(m+1)} \end{aligned}$$

All twelve spatial directions follow the same pattern with the appropriate center's displacement and exponent. $[\dots]$ means that the indices remain unchanged to the current indicies.

The algorithm. Identical strategy to NAI, scaled up: fill shell by shell in total angular momentum ' $L = 1, \dots, l_a + l_b + l_c + l_d$ ', visiting only index combinations summing to L , filling orders ' $m = 0 \dots (L_{\text{max}} - L)$ ', stepping the first nonzero spatial index (the twelve `if/elif` branches). Increasing- L order guarantees dependencies exist before use.

Index conventions and the swap helper

`swap_ERI({axis: delta, ...})` is the 13-index analog of `shift_NAI`: same shift-and-bounds-check pattern, with the axis map now spanning all twelve spatial indices ' $(a_x, a_y, a_z, b_x, b_y, b_z, c_x, c_y, c_z, d_x, d_y, d_z)$ ' plus m . This keeps the twelve step-up branches readable.

Source: [S. Obara and A. Saika, J. Chem. Phys 84, 3963 \(1986\)](#)